

COLLEGE OF ENGINEERING TRIVANDRUM
Department of Electrical and Electronics Engineering



Design and Optimisation of a Closed-Loop DC-DC Boost Converter

Using Genetic Algorithm-Tuned PI Control

Project Report

Author:

Varun Sathya
BTech EEE

April 21, 2026

1 Introduction

A DC-DC boost converter steps up a lower DC input voltage to a higher DC output voltage. In an ideal lossless converter, the output voltage is determined entirely by the input voltage and the switch duty cycle. In practice however, switching losses, diode forward voltage drops, and parasitic resistances cause the actual output to fall below the theoretical value. A fixed duty cycle cannot correct for these losses, and open loop control is therefore insufficient for precision voltage regulation.

This project addresses that limitation through three stages. First, an open loop boost converter is simulated to demonstrate the gap between theoretical and actual output voltage. Second, a closed loop PI controller is introduced to regulate the output to the 20V target by automatically adjusting the duty cycle. Third, the PI controller gains are optimised using a Genetic Algorithm (GA), a metaheuristic search method that minimises a cost function encoding the desired output behaviour. The result is a systematic, automated method for tuning a power converter controller without manual trial and error.

2 Boost Converter Parameters

The circuit was designed to step up a 12V input to a 20V output at 100W with a switching frequency of 25 kHz. The full set of design parameters is given in Table 1.

Table 1: Boost Converter Design Parameters

Parameter	Symbol	Value
Input Voltage	V_{in}	12 V
Output Voltage (Target)	V_{out}	20 V
Switching Frequency	f_s	25 kHz
Output Power	P	100 W
Duty Cycle (Nominal)	D	0.4
Output Current	I_o	5 A
Inductor Current	I_L	8.33 A
Inductor	L	76.8 μ H
Capacitor	C	220 μ F x2
Load Resistance	R	4 Ω
Ripple in Output Voltage	ΔV_o	0.2 V (1%)
Ripple in Inductor Current	ΔI_L	2.5 A (30%)

The nominal duty cycle is derived from the ideal boost converter voltage gain:

$$D = 1 - \frac{V_{in}}{V_{out}} = 1 - \frac{12}{20} = 0.4 \quad (1)$$

3 Open Loop Boost Converter

3.1 Circuit Description

The open loop converter was implemented in MATLAB Simulink using the Simscape Electrical library. The circuit consists of a 12V DC source, an inductor, an N-channel MOSFET, a diode, an output filter capacitor, and a 4Ω resistive load. The MOSFET gate is driven by a fixed-frequency PWM signal with duty cycle $D = 0.4$ and switching frequency $f_s = 25$ kHz. No feedback path is present.

3.2 Theoretical Output Voltage

For an ideal lossless boost converter operating in continuous conduction mode, the DC voltage gain is:

$$M = \frac{V_{out}}{V_{in}} = \frac{1}{1 - D} \quad (2)$$

Substituting $D = 0.4$ and $V_{in} = 12$ V:

$$V_{out} = \frac{12}{1 - 0.4} = \frac{12}{0.6} = 20 \text{ V} \quad (3)$$

3.3 Simulated Result

The simulated open loop output voltage settled at approximately **18.75 V**, which is 6.25% below the 20V target. This deficit arises from:

- Diode forward voltage drop reducing the effective output
- MOSFET on-state resistance causing conduction losses
- Inductor series resistance dissipating energy

These are inherent non-idealities that a fixed duty cycle cannot compensate for. This result confirms that closed loop feedback control is necessary to achieve the desired output voltage, and reflects what would be observed in a real physical circuit.

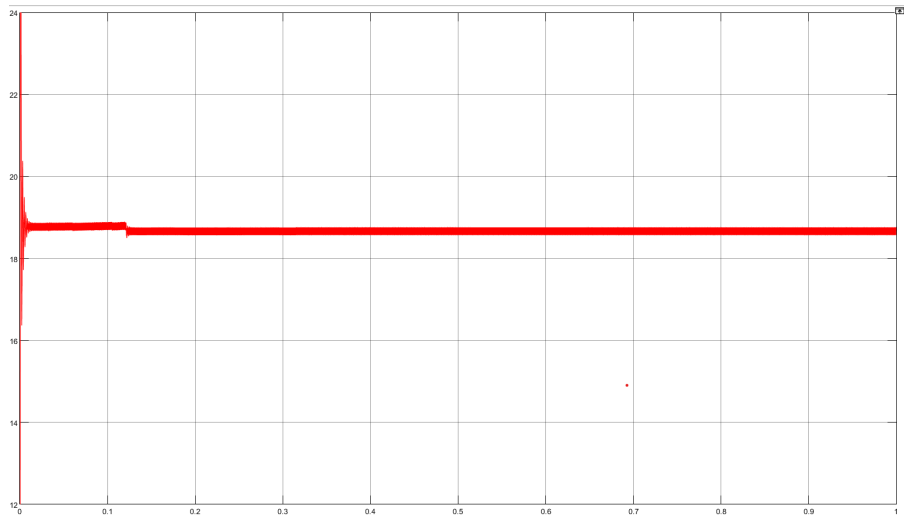


Figure 1: Graph of output voltage (V_o) of the open loop boost converter

4 Closed Loop PI Controller

4.1 Motivation

Since the open loop duty cycle of 0.4 yields only 18.75V, a higher duty cycle is needed to reach 20V. The exact duty cycle that achieves 20V under real circuit conditions is not known analytically because it depends on all the non-ideal loss mechanisms simultaneously. A PI controller solves this by continuously measuring the output voltage, computing the error from the 20V reference, and adjusting the duty cycle in real time until the error is driven to zero.

4.2 Control Loop Structure

The closed loop signal flow is as follows:

1. The output voltage V_{out} is measured using a voltage measurement block
2. The error is computed: $e(t) = V_{ref} - V_{out}(t)$, where $V_{ref} = 20$ V
3. The error is normalised by a gain of $1/V_{ref} = 1/20$
4. The PI controller processes the normalised error and outputs a duty cycle command.
The PI control law in the s-domain is:

$$C(s) = K_p + \frac{K_i}{s} \quad (4)$$

5. The duty cycle command is clamped to $[0, 1]$ and fed to a PWM comparator
6. The comparator drives the MOSFET gate signal

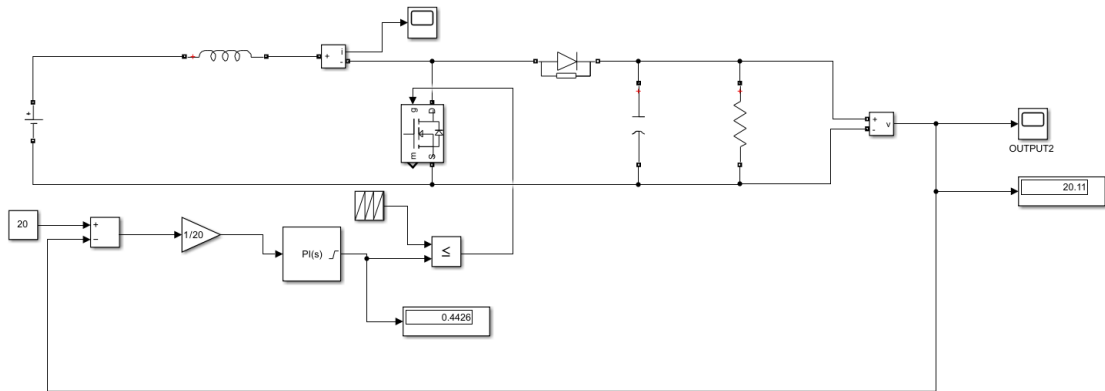


Figure 2: Closed loop boost converter using a PI controller with tuned parameters

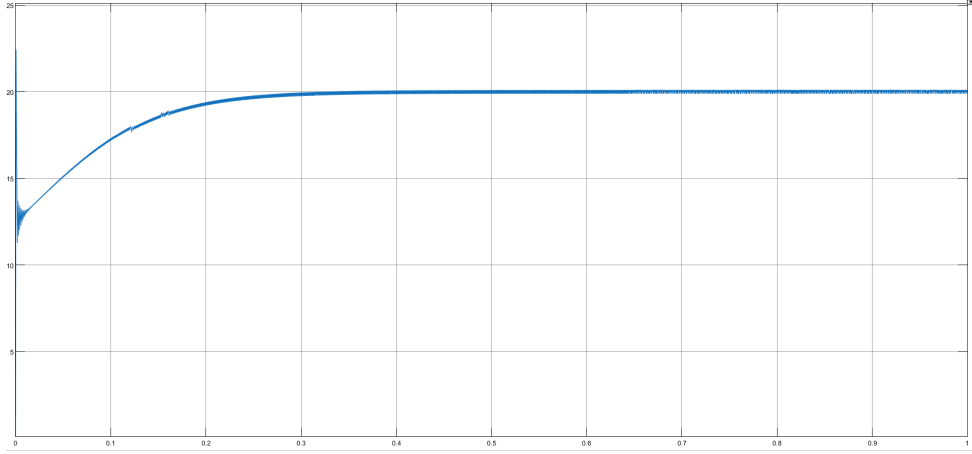


Figure 3: Graph of output voltage (V_o) for Closed loop boost coverter

4.3 Plant Transfer Function

MATLAB Simulink's built-in PID Tuner app attempts to automatically linearize the plant by computing the Jacobian of the model around an operating point. This fails for switched-mode power converter circuits because the MOSFET and diode are nonlinear switching elements.

As an initial exploration to overcome this, a small-signal plant transfer function was derived analytically to serve as a baseline model:

$$G_{vd}(s) = \frac{V_{in}}{D'^2} \cdot \frac{-Ls + RD'^2}{LCR \cdot s^2 + L \cdot s + RD'^2} \quad (5)$$

In MATLAB:

```

1 Vin = 12; Vout_ref = 20;
2 L = 76.8e-6; C = 400e-6; R = 4;
3 D_prime = Vin / Vout_ref; % = 0.6
4
5 num = (Vin / (D_prime^2)) * [-L, R*(D_prime^2)];
6 den = [(L*C*R), L, R*(D_prime^2)];
7 G_vd = tf(num, den);

```

Listing 1: Initial Small-Signal Plant Transfer Function

While this linear model provided early insights and allowed for basic initial tuning, it does not fully capture the nonlinear dynamics, switching behavior, or physical limits (such as saturation) of the real circuit. This limitation ultimately motivated the transition to a direct nonlinear **Ordinary Differential Equation (ODE) solver** for the final Genetic Algorithm cost function evaluation, ensuring a more accurate and robust optimization process.

5 Genetic Algorithm Parameter Optimisation

5.1 Motivation

Classical tuning methods such as Ziegler-Nichols produce a deterministic solution based on a linearized plant model. They do not directly optimize for time-domain criteria such as settling time or overshoot. They also break down in nonlinear or poorly modeled plants, where the linearized model no longer reflects the real system. A Genetic Algorithm is a population-based search method that mimics natural selection, and iteratively evolves candidate solutions by selecting, crossing, and mutating them to minimize a cost function. Because it searches the parameter space directly rather than relying on a plant model, it can optimize PID gains against any user-defined time-domain objective. This makes it well suited to cases where classical methods struggle.

5.2 Algorithm Overview

The Genetic Algorithm starts with a randomly generated population of candidate solutions (in this case, pairs of $[K_p, K_i]$ values). Each candidate is evaluated by the cost function. The fittest candidates are selected, crossed over to produce offspring, and randomly mutated. This repeats over multiple generations until the population converges to a near-optimal solution.

5.3 Search Space

The GA was constrained to search within bounds chosen around the expected range of valid gains:

Table 2: GA Search Bounds

Parameter	Lower Bound	Upper Bound
K_p	0.0001	1.0
K_i	1.0	100.0

5.4 Cost Function

The cost function evaluates the closed-loop system using a direct ODE simulation rather than a linearized transfer function, improving accuracy. It encodes the objective of minimizing the Integral Square Error (ISE).

The ISE criterion is defined as:

$$\text{ISE} = \int_0^{t_{\text{end}}} e(t)^2 dt \approx \sum_{k=0}^N e(t_k)^2 \cdot \Delta t \quad (6)$$

The full cost function applies hard penalties for instability or excessive overvoltages:

$$J(K_p, K_i) = \begin{cases} 10^6 & \text{if unstable or } \max(V_{\text{out}}) > 40\text{V} \\ \text{ISE} & \text{otherwise} \end{cases} \quad (7)$$

5.5 MATLAB Implementation

```

1 function [Kp_out, Ki_out] = ga_pid_tuner()
2     % =====
3     % Parallel Pooled Non-Linear Genetic Algorithm PID Tuner
4     % Called extrinsically by Simulink at t=0
5     % =====
6     rng(42);
7     disp('Simulink paused at t=0. Booting 10 parallel GA workers
8         ...');
9     num_runs = 10;
10    best_Kp = zeros(num_runs, 1);
11    best_Ki = zeros(num_runs, 1);
12    best_costs = zeros(num_runs, 1);
13    % Plant Parameters & Bounds
14    Vin = 12; Vref = 20; L = 76.8e-6; C = 400e-6; R = 4;
15    lb = [0.0001, 1.0];
16    ub = [1, 100.0];
17    % Options (UseParallel MUST be false here for parfor to work)
18    options = optimoptions('ga', ...
19        'Display', 'off', ...
20        'PopulationSize', 30, ...
21        'MaxGenerations', 50, ...
22        'UseParallel', false);
23    % Execute 10 independent runs in parallel
24    parfor i = 1:num_runs
25        [opt_K, fval] = ga(@(K) eval_boost_ode(K, Vin, Vref, L, C
26            , R), ...
27            2, [], [], [], [], lb, ub, [], options);
28        best_Kp(i) = opt_K(1);
29        best_Ki(i) = opt_K(2);
30        best_costs(i) = fval;
31        fprintf('Worker %d finished | Cost: %.4f | Kp: %.4f | Ki:
32            %.4f\n', i, fval, opt_K(1), opt_K(2));
33    end
34    % Extract the absolute best from the 10 runs
35    [absolute_best_cost, best_idx] = min(best_costs);
36    Kp_out = best_Kp(best_idx);
37    Ki_out = best_Ki(best_idx);
38    fprintf('--- GA Complete. Resuming Simulink ---\n');
39    fprintf('Global Champion (Worker %d): Kp = %f | Ki = %f (Cost
40        : %f)\n', ...
41        best_idx, Kp_out, Ki_out, absolute_best_cost);
42    end
43    % =====
44    % Cost Function
45    % =====
46    function cost = eval_boost_ode(K, Vin, Vref, L, C, R)
47        Kp = K(1); Ki = K(2);
48        x0 = [0; 0; 0];

```

```

46     tspan = [0, 0.05];
47
48     try
49         warning('off', 'all');
50         [t, x] = ode45(@(t, x) boost_dynamics(t, x, Kp, Ki, Vin,
51             Vref, L, C, R), tspan, x0);
52         warning('on', 'all');
53
54         Vout = x(:, 2);
55         error = abs(Vref - Vout);
56
57         if any(isnan(Vout)) || max(Vout) > 40 || Vout(end) < 0
58             cost = 1e6;
59             return;
60         end
61
62         % Integral Square Error
63         dt = [0; diff(t)];
64         cost = sum((error.^2) .* dt);
65     catch
66         cost = 1e6;
67     end
68 end
69 % =====
70 % Plant Dynamics: Soft-Start + Anti-Windup(clamped) + Damping
71 % =====
72 function dxdt = boost_dynamics(t, x, Kp, Ki, Vin, Vref, L, C, R)
73     iL = x(1); Vout = x(2); err_int = x(3);
74
75     Vref_dynamic = Vref * (1 - exp(-t / 0.005));
76     error = Vref_dynamic - Vout;
77     d = Kp * error + Ki * err_int;
78     derr_int_dt = error;
79
80     if d >= 0.85
81         d = 0.85;
82         if error > 0, derr_int_dt = 0; end
83     elseif d <= 0.0
84         d = 0.0;
85         if error < 0, derr_int_dt = 0; end
86     end
87
88     RL = 0.05;
89     dVout_dt = ((1 - d) * iL) / C - Vout / (R * C);
90     diL_dt = (Vin - iL * RL - (1 - d) * Vout) / L;
91
92     dxdt = [diL_dt; dVout_dt; derr_int_dt];
93 end

```

Listing 2: Parallel Genetic Algorithm PI Tuner

5.6 Simulink Integration

The GA tuner was called from a MATLAB Function block in Simulink. A persistent variable ensures the GA runs only once at the first simulation timestep, after which the stored gains are passed to the PI controller block for every subsequent timestep without recomputation.

```
1 function [Kp, Ki] = pid_tuner()
2     coder.extrinsic('pid_tuner_ga');
3
4     persistent Kp_stored Ki_stored
5     Kp = 0; Ki = 0;
6
7     if isempty(Kp_stored)
8         [Kp_stored, Ki_stored] = pid_tuner_ga();
9     end
10
11     Kp = Kp_stored;
12     Ki = Ki_stored;
13 end
```

Listing 3: MATLAB Function Block for Simulink Integration

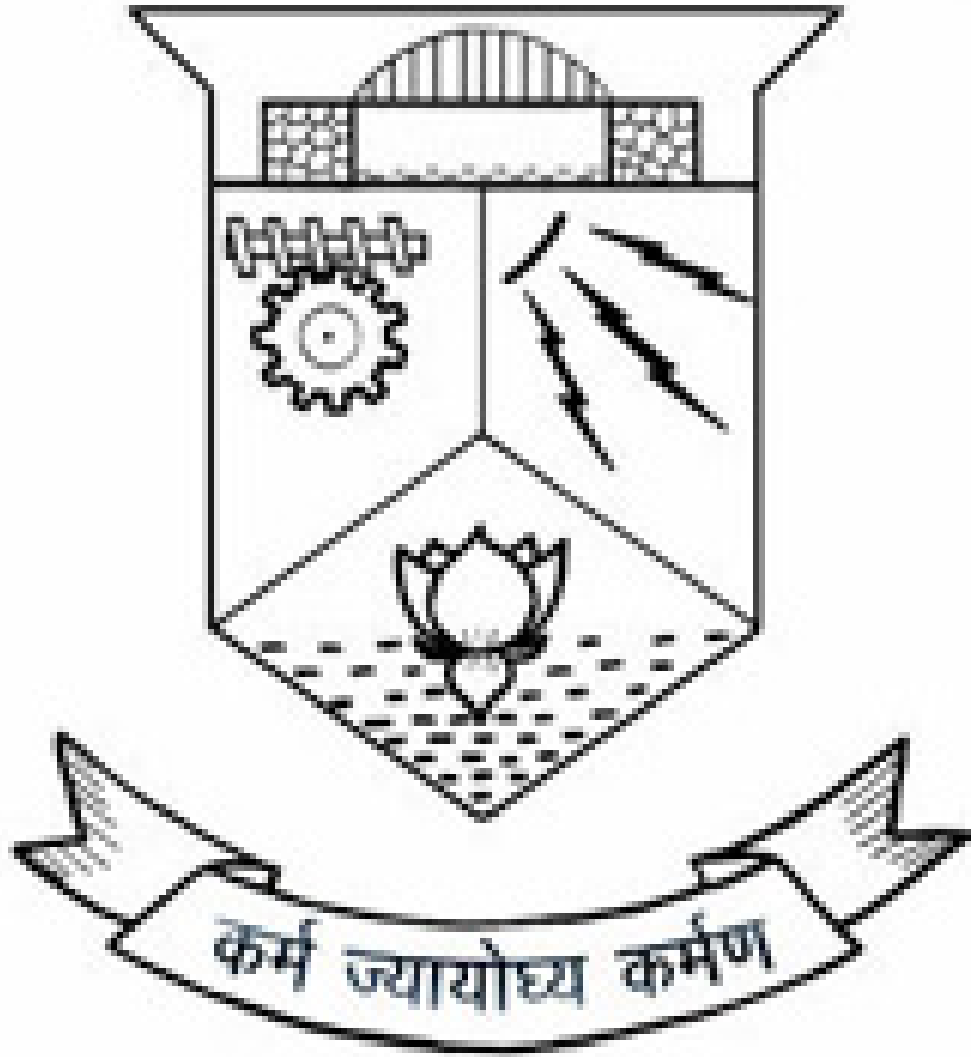


Figure 4: GA results (10 parallel pooling runs)

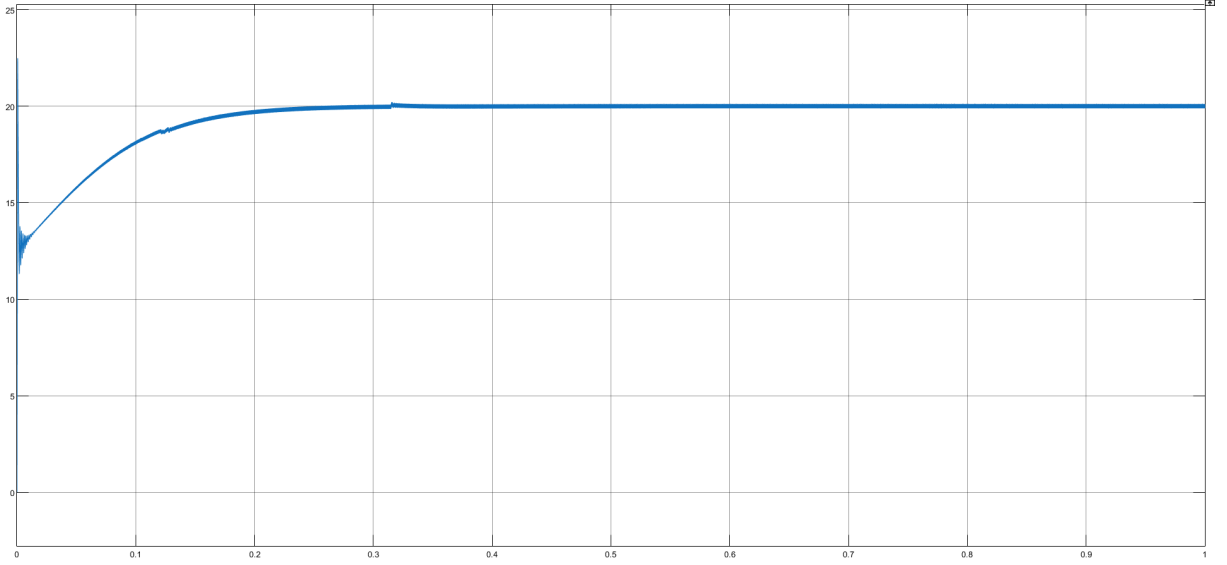


Figure 5: Output voltage (V_o) of boost coverter with genetic algorithm-based PID tuning

6 Simulation Configuration

Table 3: Simulink Simulation Settings

Setting	Value
Solver	Variable step, continuous (auto)
PID sample time	$40 \times 10^{-6} \text{ s } (= 1/f_s)$
Anti-windup method	Clamping
Output saturation	[0.1, 0.9]

The PID sample time was set to match the switching period $T_s = 1/f_s = 40 \mu\text{s}$ so the controller updates exactly once per switching cycle. Anti-windup clamping was enabled with output limits of $[0, 1]$ to prevent the integrator from accumulating beyond the physical duty cycle range.

7 Results and Discussion

7.1 Open Loop

With a fixed duty cycle of $D = 0.4$, the simulated output voltage settled at **18.75 V**. The theoretical value from the ideal boost converter formula is 20V, giving a deficit of 1.25V (6.25%). This confirms the presence of real circuit losses that open loop control cannot compensate for.

7.2 Closed Loop with GA-Optimised PI

With the GA-tuned PI controller active, the output voltage converged to approximately **20 V** with an increased duty cycle of approximately **0.4397**.

Key results:

- Steady state output voltage reached the 20V reference
- The duty cycle adjustment of +0.0397 (from 0.4 to 0.4397) compensates for circuit non-idealities automatically
- The output settled safely with the soft-start implementation
- Output ripple remained within acceptable bounds at the switching frequency

7.3 Comparison

Table 4: Performance Comparison Across Configurations

Metric	Open Loop	Closed Loop (GA PI)
Output Voltage	18.75 V	≈ 20 V
Steady State Error	1.25 V (6.25%)	≈ 0 V
Duty Cycle	0.4 (fixed)	0.4397 (automatic)
Voltage Regulation	None	Active
Tuning Method	N/A	Genetic Algorithm + ISE

8 Conclusion

This project demonstrated the full design pipeline for a closed loop DC-DC boost converter, from open loop characterisation through to GA-optimised PI control. The open loop converter confirmed that circuit losses prevent the theoretical 20V output from being achieved with a fixed duty cycle. The closed loop PI controller resolved this by adapting the duty cycle in real time. The use of a Genetic Algorithm to tune the PI gains provided a systematic, automated approach that directly optimised for time-domain output voltage performance through the ISE criterion and ODE-based dynamic simulation, with hard penalties enforcing stability. The result was a well-regulated 20V output with improved dynamics, demonstrating the effectiveness of metaheuristic optimisation for power electronics controller design.